CẨM NANG TRIỂN KHAI DÀNH CHO NGƯỜI MỚI

SÁU NHÓM KIẾN THỨC CẦN HỌC ĐỂ TRIỂN KHAI QWEN

Từ hiểu model, xây dữ liệu và QLoRA đến tích hợp Next.js production

Tài liệu học theo đầu ra: mỗi giai đoạn đều có kiến thức, bài thực hành và tiêu chí hoàn thành

Phạm vi	Thời gian tham khảo	Mục tiêu đầu tiên
Qwen 4B cho dịch Việt-Anh	6-10 tuần nếu đã biết lập trình	Self-host và benchmark trước khi train

Phiên bản 1.0 | Tháng 8 năm 2026

1. Cách sử dụng tài liệu

Không cần học hết Machine Learning. Hãy học đủ để xây một vòng đời model có thể đo lường, triển khai và rollback an toàn.

> **Nguyên tắc cốt lõi:** Chạy Qwen nguyên bản, tạo bộ test chuẩn và benchmark trước. Chỉ fine-tune khi đã xác định được nhóm lỗi lặp lại mà prompt hoặc rule không giải quyết ổn định.

Giai đoạn	Sản phẩm phải tạo ra	Câu hỏi cần trả lời
1. Model và hạ tầng	Bảng capacity planning	Cần GPU nào và chi phí bao nhiêu?
2. Kiến thức nền	Model chạy được qua REST API	Đã đủ năng lực kỹ thuật chưa?
3. Kho dữ liệu	Golden set và dataset có version	Dữ liệu có đúng và có quyền sử dụng không?
4. Self-host	Báo cáo benchmark	Model gốc đã đủ tốt chưa?
5. SFT + QLoRA	Adapter và model card	Fine-tune có tốt hơn baseline không?
6. Next.js production	Pilot có fallback	Có vận hành và rollback an toàn không?

2. Tìm hiểu model, tham số và chi phí hạ tầng

2.1. Khái niệm model bắt buộc

- Base model và Instruct model: ứng dụng hội thoại nên bắt đầu từ bản Instruct.
- Token và tokenizer: model xử lý token số, không đọc trực tiếp từ hoặc câu.
- Chat template: quy tắc biến system, user và assistant thành chuỗi token.
- Context length: tổng token đầu vào và đầu ra mà request có thể sử dụng.
- Sampling: temperature, top_p, max_tokens và stop; dịch cần cấu hình ổn định, thường temperature rất thấp.
- Precision và quantization: FP32, BF16/FP16, INT8 và INT4.
- KV cache, throughput, latency và concurrency: bốn yếu tố quyết định dung lượng phục vụ thực tế.

> **Model nên thử đầu tiên:** Qwen3-4B-Instruct-2507 là model non-thinking 4B, phù hợp để dựng baseline dịch câu ngắn. Ứng dụng không cần dùng context rất dài; bắt đầu ở khoảng 2.048-4.096 token để giảm KV cache và rủi ro OOM.

2.2. Ước lượng VRAM

Bộ nhớ thô của trọng số được ước lượng bằng số tham số nhân số byte cho mỗi tham số. Runtime thực tế còn cần KV cache, CUDA, tensor trung gian và bộ nhớ dự phòng.

Định dạng	Trọng số thô của model 4B	Ứng dụng

Định dạng	Trọng số thô của model 4B	Ứng dụng
FP32	Khoảng 16 GB	Không phù hợp để inference tiết kiệm
BF16 / FP16	Khoảng 8 GB	Chất lượng tốt, cần VRAM cao hơn
INT8	Khoảng 4 GB	Giảm bộ nhớ
INT4	Khoảng 2 GB	Phù hợp thử nghiệm và QLoRA

Mục đích	GPU/VRAM khởi đầu	Lưu ý
Thử inference INT4	6-8 GB	Concurrency thấp
Self-host context ngắn	12-16 GB	Benchmark P95 và KV cache
Pilot QLoRA	24 GB	Điểm khởi đầu thực hành
LLM + ASR + TTS	24 GB trở lên	Nên tách service và đo tải

2.3. Các nhóm chi phí

- GPU, CPU, RAM, SSD và băng thông.
- Điện, làm mát hoặc giá thuê GPU theo giờ.
- Lưu model, checkpoint, dataset, log và backup.
- Monitoring, bảo mật, thời gian kỹ sư và máy dự phòng.
- Chi phí do timeout, downtime, fallback và GPU nhàn rỗi.

> **Tiêu chí hoàn thành:** Có bảng GPU/VRAM, request đồng thời, độ dài prompt/output, token/giây, latency P50/P95, tỷ lệ OOM và chi phí trên 1.000 request.

3. Xác định kiến thức cần học và phần có thể bỏ qua

3.1. Nền tảng bắt buộc

Nhóm	Nội dung phải biết
Python	Hàm, class, exception, JSON/JSONL/CSV, virtual environment và package
Linux	File, process, quyền, biến môi trường, log, CPU/RAM/GPU
Git	Branch, commit, tag và version code/dataset/config
Docker	Image, container, volume, port, health check và NVIDIA runtime
HTTP/REST	JSON, authentication, timeout, retry, streaming và status code
AI	Tokenizer, loss, learning rate, epoch, batch, split, overfitting, quantization

Nhóm	Nội dung phải biết
Ứng dụng	TypeScript, Next.js server-side, schema validation, log và feature flag

3.2. Có thể tạm bỏ qua

- Pretraining Transformer và tokenizer từ đầu.
- Tự viết CUDA kernel hoặc tối ưu compiler.
- Distributed training nhiều node, NVLink và InfiniBand.
- RLHF, reward model, DPO và GRPO.
- Mixture-of-Experts chuyên sâu.
- Tự train ASR, TTS hoặc voice cloning cùng lúc với LLM.

> **Tiêu chí hoàn thành:** Chạy được model Hugging Face bằng Python; dựng container GPU; gọi model qua REST; giải thích được token, context, loss và overfitting; đọc được log GPU và lỗi OOM.

4. Xây kho dữ liệu Việt-Anh có kiểm duyệt

Chất lượng dữ liệu và test set quyết định giá trị của fine-tuning. Audio, transcript và bản dịch là ba loại dữ liệu khác nhau.

4.1. Dữ liệu cho từng model

Model	Đầu vào	Nhãn chuẩn
ASR / Whisper	Audio tiếng Việt	Transcript tiếng Việt
LLM / Qwen	Câu tiếng Việt	Bản dịch tiếng Anh
TTS	Văn bản tiếng Anh	Audio tiếng Anh tương ứng

> **Lưu ý:** Không đưa trực tiếp audio vào Qwen text model. Muốn train Qwen phải có transcript tiếng Việt và bản dịch tiếng Anh đã được con người xác nhận.

4.2. Schema và metadata

```
{
  "id": "vi_en_000001",
  "vietnamese": "Mẹ ơi, con không muốn mua cái này.",
  "english": "Mom, I don't want to buy this.",
  "context": "outside",
  "region": "south",
  "speech_act": "refusal",
  "review_status": "human_approved",
  "split": "train"
}
```

- Lưu nguồn dữ liệu, reviewer, thời gian duyệt và phiên bản guideline.
- Dùng speaker_id đã ẩn danh thay cho tên thật.
- Gắn nhãn phủ định, chủ thể, người nghe, số lượng, màu sắc, thời gian và địa điểm.
- Lưu consent version và chính sách lưu/xóa dữ liệu nếu có dữ liệu trẻ em.

4.3. Quy trình kiểm duyệt

1. Người thứ nhất tạo hoặc sửa bản dịch.
2. Người thứ hai kiểm tra độc lập.
3. Bất đồng được chuyển cho người có quyền quyết định.
4. Ghi lại lịch sử sửa và lý do sửa.
5. Audit ngẫu nhiên 5-10% dữ liệu đã duyệt.

4.4. Chia tập và quy mô pilot

Tập	Tỷ lệ	Quy tắc
Train	80%	Dùng cập nhật adapter
Validation	10%	Chọn checkpoint và phát hiện overfitting
Test	10%	Khóa kín, không dùng chọn tham số

- Chia audio theo người nói; không để cùng một trẻ xuất hiện ở train và test.
- Gom câu gần trùng theo cụm ngữ nghĩa trước khi chia.
- Bắt đầu với 500-1.000 mẫu golden test và 2.000-5.000 mẫu SFT có human review.
- Không dùng output Cloudflare chưa kiểm duyệt làm đáp án chuẩn.

> **Tiêu chí hoàn thành:** Có guideline, dataset được version, báo cáo chất lượng, golden test đã khóa và danh sách nguồn/consent.

5. Chạy Qwen self-host và benchmark trước khi train

5.1. Kiến trúc pilot

```
Next.js backend
    ↓ HTTPS + API key
Qwen model service riêng
    ↓
vLLM + GPU + Qwen3-4B-Instruct-2507
```

- Không nạp model trực tiếp vào tiến trình Next.js hoặc serverless function.
- Khóa revision model, tokenizer, vLLM, PyTorch, CUDA và Docker image.
- Bắt đầu max-model-len khoảng 4.096 cho câu dịch ngắn.
- Đặt temperature rất thấp và giới hạn max_tokens.

- Bảo vệ endpoint bằng HTTPS, API key, firewall và network policy.

5.2. Ma trận benchmark

Ứng viên	Mục đích	Điều kiện đánh giá
Cloudflare	Baseline hiện tại	Cùng prompt và cùng golden test
Qwen nguyên bản	Xác định có cần train	Không kém đáng kể ở lỗi trọng yếu
Qwen + prompt/rule	Tối ưu không train	Ổn định hơn model gốc
Qwen + LoRA	Ứng viên production	Tốt hơn baseline mà không overfit

5.3. Chỉ số phải đo

- Giữ đúng phủ định, người nói/người nghe và quan hệ gia đình.
- Không mất màu sắc, số lượng, thời gian hoặc địa điểm.
- Không tự thêm nghĩa, lịch sự hoặc hoàn chỉnh câu thiếu.
- Đúng định dạng: chỉ trả một câu tiếng Anh nếu ứng dụng yêu cầu.
- Human acceptance rate, latency P50/P95, token/giây, timeout, OOM và chi phí.
- Load test ở nhiều mức concurrency và độ dài request.

> **Quyết định go/no-go:** Chỉ fine-tune khi lỗi lặp lại, prompt/rule không giải quyết ổn định, có đủ ví dụ chất lượng và golden test đo được mức cải thiện.
>
> **Tiêu chí hoàn thành:** Có model server OpenAI-compatible, báo cáo Cloudflare-vs-Qwen, danh sách nhóm lỗi và quyết định có cần fine-tune hay không.

6. Fine-tune bằng SFT + QLoRA

6.1. Cơ chế

```
Qwen base nạp 4-bit và được đóng băng
                +
LoRA adapter nhỏ được train ở precision cao hơn
                ↓
Base model + adapter = model theo miền ứng dụng
```

- SFT học từ cặp câu đầu vào và câu trả lời chuẩn.
- LoRA chỉ cập nhật ma trận low-rank, giữ nguyên trọng số gốc.
- QLoRA nạp base model ở 4-bit để giảm VRAM.
- Adapter nhỏ, dễ lưu version, phân phối và rollback.

6.2. Định dạng hội thoại

```
{
  "messages": [
    {"role": "system", "content": "Translate Vietnamese to English faithfully."},
    {"role": "user", "content": "Con không muốn đi học."},
    {"role": "assistant", "content": "I don't want to go to school."}
  ]
}
```

> **Loss:** Ưu tiên tính loss trên phần assistant/completion để model học câu trả lời chuẩn, không học lại toàn bộ prompt. Phải kiểm tra chat template tương thích.

6.3. Tham số pilot tham khảo

Tham số	Điểm khởi đầu	Mục đích
Quantization	NF4 4-bit	Giảm VRAM
LoRA rank	16 hoặc 32	Bắt đầu nhỏ rồi benchmark
LoRA alpha	16-32	Điều khiển mức scaling
LoRA dropout	Khoảng 0,05	Hạn chế overfitting
Sequence length	512-1.024	Phù hợp câu ngắn
Epoch	2-3	Dừng sớm nếu validation xấu đi
Effective batch	16-32	Dùng gradient accumulation
Compute dtype	BF16 nếu hỗ trợ	Ổn định và hiệu quả

6.4. Trình tự một lần train

6. Khóa version base model, tokenizer, dataset và training config.
7. Kiểm tra schema, trùng lặp, PII và split.
8. Chạy baseline trên test set trước khi train.
9. Train adapter và theo dõi training/validation loss.
10. Chọn checkpoint theo validation, không chọn theo cảm giác.
11. Chạy test kín đúng một quy trình đã định trước.
12. Human review và phân tích theo từng nhóm lỗi.
13. Lưu adapter, model card, config và báo cáo.

> **Tiêu chí hoàn thành:** Có adapter, biểu đồ loss, dataset manifest, training config và báo cáo chứng minh model tốt hơn baseline mà không overfit.

7. Tích hợp backend Next.js và triển khai production

7.1. Provider abstraction

```
translateVietnameseToEnglish(input, options)
    ├─ cloudflare
    └─ self_hosted → OpenAI-compatible API
```

- Giữ interface chung để phần còn lại của ứng dụng không phụ thuộc trực tiếp vLLM.
- Tách provider, prompt builder, output normalization và telemetry.
- Thêm feature flag để đổi provider mà không thay toàn bộ luồng ứng dụng.

7.2. Biến môi trường và bảo mật

```
TEXT_AI_PROVIDER=self_hosted
SELF_HOSTED_LLM_BASE_URL=https://gpu.example.com/v1
SELF_HOSTED_LLM_API_KEY=<secret>
SELF_HOSTED_LLM_MODEL=Qwen/Qwen3-4B-Instruct-2507
SELF_HOSTED_LLM_TIMEOUT_MS=2500
```

- API key chỉ tồn tại server-side; không dùng tiền tố NEXT_PUBLIC_.
- Endpoint GPU dùng HTTPS, firewall, allowlist và quyền truy cập tối thiểu.
- Không log nguyên văn dữ liệu nhạy cảm; log request ID và metadata vận hành.
- Xác thực người dùng trước Route Handler nếu endpoint có thể bị gọi từ bên ngoài.

7.3. Timeout, fallback và kiểm soát đầu ra

```
Cache → Qwen self-host → kiểm tra schema/fidelity
              ↓ timeout, OOM hoặc output lỗi
              Cloudflare fallback
```

- Giới hạn input, max_tokens, timeout và request size.
- Chỉ retry tối đa một lần có kiểm soát.
- Dùng queue, concurrency limit và circuit breaker.
- Xác thực schema, độ dài và nội dung bắt buộc trước khi trả cho người dùng.
- Ghi model version, adapter version, latency và fallback reason.

7.4. Rollout

```
Offline benchmark → Shadow traffic → Canary 5% → 25% → 50% → 100%
```

- Shadow: Qwen chạy song song nhưng Cloudflare vẫn trả kết quả.
- Canary: chỉ một tỷ lệ nhỏ request dùng Qwen.
- Tăng tỷ lệ khi chất lượng, latency và error rate đạt ngưỡng.
- Giữ feature flag và rollback adapter/model trong suốt rollout.

7.5. Monitoring và kiểm thử

Nhóm	Cần theo dõi/kiểm thử
Chất lượng	Acceptance rate, lỗi phủ định, fidelity và output không hợp lệ
Hiệu năng	P50/P95/P99, token/giây, queue depth và concurrency
GPU	Utilization, VRAM, KV cache, OOM và nhiệt độ
Độ tin cậy	Timeout, fallback, health/readiness và circuit breaker
Kiểm thử	Unit, contract, integration, load, failure injection và end-to-end

> **Tiêu chí hoàn thành:** Có pilot production với provider self-host, feature flag, fallback, dashboard, cảnh báo và quy trình rollback đã được diễn tập.

8. Lộ trình thực hiện đề xuất

Tuần	Trọng tâm	Đầu ra
1	Model, token, GPU và chi phí	Bảng capacity planning
2	Python, Linux, Docker và REST	Model chạy qua API
3-4	Golden dataset và guideline	500-1.000 mẫu đã duyệt
5	Self-host và benchmark	Báo cáo go/no-go cho fine-tune
6-7	SFT + QLoRA	Adapter và model card
8-10	Next.js, shadow, canary	Pilot production có fallback

> **Bước tiếp theo:** Dựng Qwen nguyên bản và benchmark trên 500 câu chuẩn. Đây là bước tạo nhiều thông tin nhất với chi phí và rủi ro thấp hơn so với train ngay.

9. Tài liệu học chính thức

- Qwen3-4B-Instruct-2507 - model card và hướng dẫn chạy
- Qwen với vLLM - OpenAI-compatible model server
- Hugging Face LLM Course - tokenizer, Transformer và fine-tuning
- TRL SFTTrainer - SFT dataset hội thoại
- TRL PEFT/QLoRA - 4-bit quantization và adapter
- PEFT LoRA - cơ chế và tham số LoRA
- Next.js production checklist - bảo mật và vận hành production

Ghi chú: Các con số VRAM, thời gian và hyperparameter là điểm khởi đầu thực hành. Kết quả cuối cùng phải được benchmark trên GPU, dữ liệu và mức concurrency thực tế của dự án.